



Subtopic: Pipelining of Processor – Issues and Hurdles

1. Pipelining in Processors

Definition

Pipelining is a technique in processor design where instruction execution is divided into multiple stages (e.g., *Fetch* → *Decode* → *Execute* → *Memory Access* → *Write Back*), allowing multiple instructions to be processed simultaneously in an assembly-line fashion.

- Increases **instruction throughput** (number of instructions executed per unit time).
 - Does not reduce the execution time of a single instruction, but multiple instructions overlap in execution.
-

2. Stages of a Typical Instruction Pipeline

1. **Instruction Fetch (IF)**: Get instruction from memory.
 2. **Instruction Decode (ID)**: Decode instruction and identify operands.
 3. **Execute (EX)**: Perform the required operation in ALU.
 4. **Memory Access (MEM)**: Access memory if needed (load/store).
 5. **Write Back (WB)**: Store result in register.
-

3. Issues and Hurdles in Pipelining



Although pipelining improves performance, it faces several challenges known as **pipeline hazards**.

3.1 Structural Hazards

- **Cause:** Occur when hardware resources are insufficient to handle multiple instructions simultaneously.
 - **Example:** If the CPU has only one memory unit, simultaneous instruction fetch and data access may conflict.
 - **Solution:**
 - Duplicate resources (separate instruction and data caches).
 - Use efficient resource allocation.
-

3.2 Data Hazards

- **Cause:** Occur when instructions depend on the results of previous instructions still in the pipeline.

Types of Data Hazards:

1. RAW (Read After Write):

- Instruction needs a value that has not yet been written.

Example:

I1: ADD R1, R2, R3

I2: SUB R4, R1, R5 ← depends on R1 result

○



- Solution: *Forwarding/Bypassing, Pipeline stalls.*

2. WAR (Write After Read):

- Instruction writes to a register before a previous instruction has read it.
- Rare in simple pipelines.

3. WAW (Write After Write):

- Two instructions write to the same register in different orders.
- Common in superscalar processors.

3.3 Control Hazards (Branch Hazards)

- **Cause:** Occur when the pipeline makes wrong decisions about instruction flow due to branch instructions (e.g., *if*, *loop*).
- **Problem:** The next instruction address is not known until the branch is resolved.
- **Solution:**
 - *Branch Prediction:* CPU guesses the outcome of the branch.
 - *Delayed Branching:* Execute next instruction regardless of branch.
 - *Speculative Execution:* Fetch and execute multiple possible paths.

3.4 Pipeline Stalls (Bubbles)

- **Definition:** Idle cycles inserted into the pipeline to resolve hazards.



- **Problem:** Reduce efficiency and throughput.
 - **Solution:** Advanced techniques like *forwarding, prediction, and out-of-order execution*.
-

3.5 Other Challenges

- **Exception Handling:** Interrupts and exceptions can disrupt normal pipeline flow.
 - **Instruction Dependencies:** Loops and frequent branching reduce pipeline effectiveness.
 - **Pipeline Depth:** Deeper pipelines improve clock speed but increase penalty on hazards (e.g., Intel Pentium 4 with 20+ stages suffered from misprediction penalties).
-

4. Techniques to Overcome Pipeline Issues

- **Operand Forwarding (Bypassing):** Use result directly from pipeline stage instead of waiting for write-back.
 - **Dynamic Scheduling:** Reorder instructions to avoid stalls.
 - **Out-of-Order Execution:** Execute independent instructions before dependent ones finish.
 - **Superscalar Design:** Multiple instructions issued per cycle.
 - **Branch Prediction Algorithms:** Static prediction, dynamic (history-based) prediction.
-



✓ Summary

- **Pipelining** increases instruction throughput by overlapping execution stages.
- Major hurdles: **Structural hazards, Data hazards, Control hazards.**
- Techniques like **forwarding, dynamic scheduling, out-of-order execution, and branch prediction** are used to minimize stalls and improve efficiency.
- While pipelining boosts performance, its effectiveness depends on smart **hazard management.**